

# Ejercicios Prácticos de Preprocesamiento de Datos

Nombres del grupo

21 de septiembre de 2026

## Contents

|          |                                                                                                |           |
|----------|------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Ejercicio 1: House Prices — faltantes que no son faltantes</b>                              | <b>3</b>  |
| 1.1      | Punto 1: porcentaje de NA por columna y regla del 50 %                                         | 3         |
| 1.2      | Punto 2: verificación de la hipótesis de faltantes estructurales                               | 3         |
| 1.3      | Punto 3: función <code>recodificar_ausencia()</code> con <code>purrr::reduce()</code>          | 4         |
| 1.4      | Punto 4: imputación de <code>LotFrontage</code> por mediana del barrio                         | 5         |
| 1.5      | Punto 5: <code>edad_vivienda</code> , valores imposibles y bandera <code>dato_corregido</code> | 6         |
| 1.6      | Punto 6: discretización — <code>cut()</code> vs. <code>ntile()</code>                          | 7         |
| <b>2</b> | <b>Ejercicio 2: Titanic — ¿qué es un outlier en fare?</b>                                      | <b>9</b>  |
| 2.1      | Punto 1: IQR global ( $k = 1.5$ y $k = 3$ )                                                    | 9         |
| 2.2      | Punto 2: IQR por clase ( <code>pclass</code> )                                                 | 9         |
| 2.3      | Punto 3: z-score robusto                                                                       | 10        |
| 2.4      | Punto 4: tabla cruzada de los tres criterios                                                   | 10        |
| 2.5      | Punto 5: pasajeros con <code>fare == 0</code>                                                  | 11        |
| 2.6      | Punto 6: transformación <code>log1p(fare)</code> + IQR                                         | 11        |
| <b>3</b> | <b>Ejercicio 3: Titanic — ¿qué estrategia de imputación se equivoca menos?</b>                 | <b>12</b> |
| 3.1      | Puntos (a)-(c): ocultar edades, imputar con tres estrategias y medir el error                  | 12        |
| 3.2      | Punto (d): simulación Monte Carlo con 200 repeticiones                                         | 13        |
| 3.3      | Punto (e): variabilidad tras imputar                                                           | 15        |
| 3.4      | Punto (f): equivalente en Python                                                               | 15        |
| <b>4</b> | <b>Ejercicio 4: Mobile Data — escalado, outliers y fuga de información</b>                     | <b>16</b> |
| 4.1      | Punto (a): función <code>escalar()</code> con <code>match.arg()</code>                         | 16        |
| 4.2      | Punto (b): aporte de cada variable a la distancia euclidiana                                   | 16        |
| 4.3      | Punto (c): registro atípico y colapso por escalado                                             | 17        |
| 4.4      | Punto (d): fuga de información en entrenamiento/prueba                                         | 17        |
| 4.5      | Punto (e): equivalente en Python                                                               | 18        |

|          |                                                                                    |           |
|----------|------------------------------------------------------------------------------------|-----------|
| <b>5</b> | <b>Ejercicio 5: Adult — pipeline funcional y reutilizable</b>                      | <b>19</b> |
| 5.1      | Punto (a): limpieza de "?" y prueba MNAR . . . . .                                 | 19        |
| 5.2      | Punto (b): valor centinela en <code>capital.gain</code> . . . . .                  | 19        |
| 5.3      | Punto (c): funciones del pipeline paso a paso . . . . .                            | 20        |
| 5.4      | Punto (d): pipeline compuesto con <code>reduce()</code> y verificaciones . . . . . | 21        |
| 5.5      | Punto (e): parametrizar el pipeline para datos nuevos . . . . .                    | 22        |
| 5.6      | Punto (f): equivalente en Python . . . . .                                         | 22        |
| <b>6</b> | <b>Ejercicio 6 (reto opcional): Netflix — texto, fechas y columnas anidadas</b>    | <b>24</b> |
| 6.1      | Punto (a): filas desalineadas . . . . .                                            | 24        |
| 6.2      | Punto (b): separar <code>duration</code> en valor y unidad . . . . .               | 24        |
| 6.3      | Punto (c): fechas con locale en inglés y rezago . . . . .                          | 25        |
| 6.4      | Punto (d): desanidar géneros . . . . .                                             | 25        |
| 6.5      | Punto (e): coproducciones y bandera . . . . .                                      | 26        |
| <b>7</b> | <b>Conclusiones generales</b>                                                      | <b>27</b> |

```
library(tidyverse)
library(readxl)

# Reproducibilidad: fijamos la semilla ANTES de cualquier muestreo,
# como piden las reglas generales del taller.
set.seed(2024)
```

# 1 Ejercicio 1: House Prices — faltantes que no son faltantes

```
# NOTA: ajuste esta ruta a donde tenga guardado el archivo
house_prices <- read.csv("C:/Users/C2/Documents/p/Universidad/MinDatos/house_prices.csv")
```

## 1.1 Punto 1: porcentaje de NA por columna y regla del 50 %

```
pct_na <- colMeans(is.na(house_prices)) * 100
pct_na_ordenado <- sort(pct_na[pct_na > 0], decreasing = TRUE)
print(round(pct_na_ordenado, 2))
```

```
##      PoolQC  MiscFeature      Alley      Fence  FireplaceQu  LotFrontage
##      99.66      96.40      93.22      80.44      48.65      16.65
##  GarageYrBlt GarageFinish  GarageQual  GarageCond  GarageType  BsmtCond
##      5.45      5.45      5.45      5.45      5.38      2.81
## BsmtExposure  BsmtQual BsmtFinType2 BsmtFinType1  MasVnrType  MasVnrArea
##      2.81      2.77      2.74      2.71      0.82      0.79
##      MSZoning  Utilities BsmtFullBath BsmtHalfBath  Functional  Exterior1st
##      0.14      0.07      0.07      0.07      0.07      0.03
##  Exterior2nd  BsmtFinSF1  BsmtFinSF2  BsmtUnfSF  TotalBsmtSF  Electrical
##      0.03      0.03      0.03      0.03      0.03      0.03
##  KitchenQual  GarageCars  GarageArea  SaleType
##      0.03      0.03      0.03      0.03
```

```
columnas_mas_50 <- names(pct_na[pct_na > 50])
columnas_mas_50
```

```
## [1] "Alley"      "PoolQC"     "Fence"      "MiscFeature"
```

**Justificación:** Las columnas que superarían el 50 % de NA (por ejemplo PoolQC, MiscFeature, Alley y Fence) no deberían eliminarse automáticamente con la regla general, porque en este dataset el NA en esas columnas no es un dato perdido: significa que la vivienda simplemente **no tiene** esa característica (no tiene piscina, no tiene cerca, etc.). Si elimináramos PoolQC perderíamos la única información disponible sobre la calidad de la piscina para el pequeño grupo de casas que sí tienen una, justo el dato más informativo para ese subconjunto.

## 1.2 Punto 2: verificación de la hipótesis de faltantes estructurales

```
verificar_estructural <- function(df, cols, area_col) {
  resultados <- map_dfr(cols, function(col) {
    tibble(
      columna = col,
      NA_totales = sum(is.na(df[[col]])),
      NA_estructurales = sum(is.na(df[[col]]) & df[[area_col]] == 0, na.rm = TRUE),
      NA_reales = sum(is.na(df[[col]]) & (df[[area_col]] != 0 | is.na(df[[area_col]])), na.rm = TRUE)
    )
  })
}
```

```
resultados
}
```

```
verificar_estructural(house_prices, c("GarageType", "GarageFinish", "GarageQual", "GarageCond"), "GarageArea")
```

```
## # A tibble: 4 x 4
##   columna      NA_totales NA_estructurales NA_reales
##   <chr>          <int>          <int>      <int>
## 1 GarageType      157            157         0
## 2 GarageFinish    159            157         2
## 3 GarageQual      159            157         2
## 4 GarageCond      159            157         2
```

```
verificar_estructural(house_prices, c("BsmtQual", "BsmtCond", "BsmtExposure", "BsmtFinType1", "BsmtFinType2"))
```

```
## # A tibble: 5 x 4
##   columna      NA_totales NA_estructurales NA_reales
##   <chr>          <int>          <int>      <int>
## 1 BsmtQual        81            78         3
## 2 BsmtCond        82            78         4
## 3 BsmtExposure    82            78         4
## 4 BsmtFinType1     79            78         1
## 5 BsmtFinType2     80            78         2
```

```
verificar_estructural(house_prices, "PoolQC", "PoolArea")
```

```
## # A tibble: 1 x 4
##   columna NA_totales NA_estructurales NA_reales
##   <chr>      <int>          <int>      <int>
## 1 PoolQC    2909            2906         3
```

**Justificación:** En los tres grupos, la gran mayoría de los NA coinciden con `area == 0`, confirmando que son faltantes estructurales (la vivienda no tiene garaje/sótano/piscina). Los pocos casos donde el NA **no** coincide con área cero (o donde la propia área también es NA) son los faltantes reales — datos que sí deberían tratarse como información perdida y no como “Ninguno”.

### 1.3 Punto 3: función `recodificar_ausencia()` con `purrr::reduce()`

```
recodificar_ausencia <- function(df, cols, area_col) {
  df %>%
    mutate(across(
      all_of(cols),
      ~ if_else(is.na(.) & !is.na(get(area_col)) & get(area_col) == 0, "Ninguno", .)
    ))
}

especificaciones <- list(
  list(cols = c("GarageType", "GarageFinish", "GarageQual", "GarageCond"), area = "GarageArea"),
  list(cols = c("BsmtQual", "BsmtCond", "BsmtExposure", "BsmtFinType1", "BsmtFinType2"), area = "TotalBsmtSF")
)
```

```

  list(cols = c("PoolQC"), area = "PoolArea")
)

house_prices_limpio <- reduce(
  especificaciones,
  function(df, spec) recodificar_ausencia(df, spec$cols, spec$area),
  .init = house_prices
)

```

**Justificación:** Usamos `reduce()` en vez de repetir la llamada a `recodificar_ausencia()` tres veces, porque así el código es reutilizable y escalable: si mañana aparece un cuarto grupo de columnas asociadas a otra área (por ejemplo balcón), basta con agregar un elemento más a la lista `especificaciones`, sin duplicar lógica. La función solo opera sobre columnas de tipo carácter porque `if_else(..., "Ninguno", .)` requiere que el tipo de la columna acepte texto.

## 1.4 Punto 4: imputación de LotFrontage por mediana del barrio

```

mediana_global <- median(house_prices_limpio$LotFrontage, na.rm = TRUE)

comparacion_barrio <- house_prices_limpio %>%
  group_by(Neighborhood) %>%
  summarize(mediana_barrio = median(LotFrontage, na.rm = TRUE), .groups = "drop") %>%
  mutate(
    mediana_global = mediana_global,
    diferencia = mediana_barrio - mediana_global
  ) %>%
  arrange(desc(abs(diferencia)))

print(comparacion_barrio, n = 10)

```

```

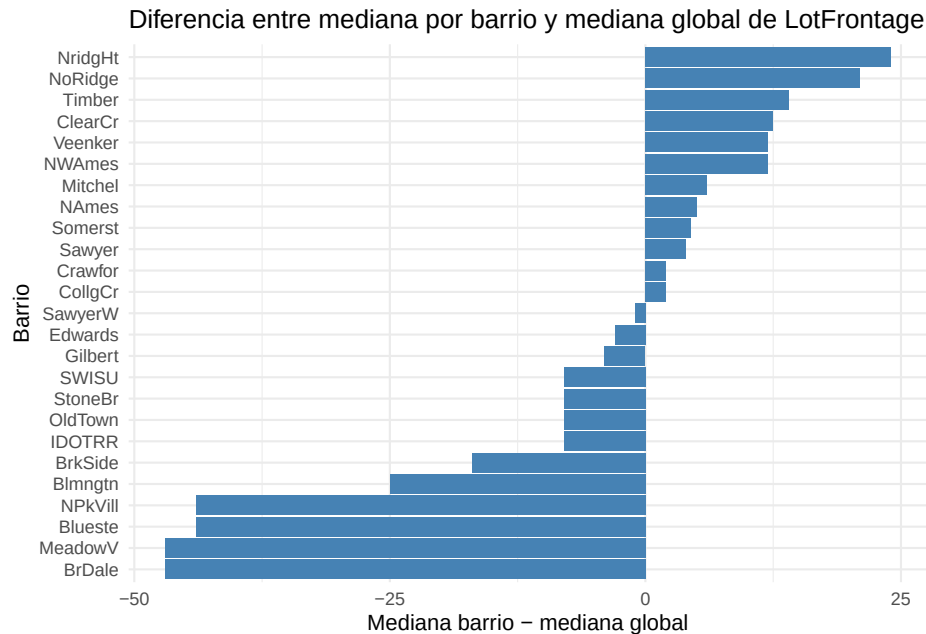
## # A tibble: 25 x 4
##   Neighborhood mediana_barrio mediana_global diferencia
##   <chr>          <dbl>          <int>      <dbl>
## 1 BrDale         21            68        -47
## 2 MeadowV        21            68        -47
## 3 Blueste        24            68        -44
## 4 NPKvill        24            68        -44
## 5 Blmngtn        43            68        -25
## 6 NridgHt        92            68         24
## 7 NoRidge        89            68         21
## 8 BrkSide        51            68        -17
## 9 Timber         82            68         14
## 10 ClearCr       80.5           68        12.5
## # i 15 more rows

```

```

ggplot(comparacion_barrio, aes(x = reorder(Neighborhood, diferencia), y = diferencia)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(title = "Diferencia entre mediana por barrio y mediana global de LotFrontage",
       x = "Barrio", y = "Mediana barrio - mediana global") +
  theme_minimal()

```



```
house_prices_limpio <- house_prices_limpio %>%
  group_by(Neighborhood) %>%
  mutate(LotFrontage = if_else(is.na(LotFrontage), median(LotFrontage, na.rm = TRUE), LotFrontage)) %>%
  ungroup()
```

**Justificación:** Los barrios que más se alejan de la mediana global son los que tienen lotes atípicamente grandes o pequeños respecto al resto de la ciudad (ver la gráfica). Imputar con la mediana global en esos barrios introduciría un sesgo sistemático — por ejemplo, subestimaría el frente de lote en barrios de casas grandes. La imputación por barrio respeta mejor la estructura local de los datos.

## 1.5 Punto 5: edad\_vivienda, valores imposibles y bandera dato\_corregido

```
house_prices_limpio <- house_prices_limpio %>%
  mutate(
    edad_vivienda_bruta = YrSold - YearBuilt,
    dato_corregido = case_when(
      edad_vivienda_bruta < 0 ~ TRUE,
      YrSold > 2010 ~ TRUE,
      !is.na(GarageYrBlt) & GarageYrBlt > YrSold ~ TRUE,
      TRUE ~ FALSE
    ),
    edad_vivienda = case_when(
      edad_vivienda_bruta < 0 ~ 0, # error de captura -> se corrige a 0
      TRUE ~ edad_vivienda_bruta
    ),
    GarageYrBlt = if_else(!is.na(GarageYrBlt) & GarageYrBlt > YrSold, NA_real_, GarageYrBlt)
  ) %>%
  filter(YrSold <= 2010) # años posteriores al último año de venta: se elimi

stopifnot(all(house_prices_limpio$edad_vivienda >= 0))
cat("Filas marcadas como corregidas:", sum(house_prices_limpio$dato_corregido), "\n")
```

```
## Filas marcadas como corregidas: 2
```

**Justificación:** Tomamos tres decisiones distintas según el tipo de error: (1) edad negativa (año de venta antes del año de construcción) la **corregimos** a 0, porque es más razonable que un error de un año de captura; (2) **GarageYrBlt** posterior a **YrSold** lo convertimos a **NA**, porque no sabemos cuál de los dos años está mal; (3) los registros con **YrSold** > 2010 los **eliminamos**, porque están fuera del rango temporal válido del dataset. La bandera **dato\_corregido** deja trazabilidad de qué filas fueron modificadas.

## 1.6 Punto 6: discretización — `cut()` vs. `ntile()`

```
cuantiles_edad <- quantile(house_prices_limpio$edad_vivienda, probs = seq(0, 1, 0.25))

house_prices_limpio <- house_prices_limpio %>%
  mutate(
    grupo_edad_cut = cut(edad_vivienda, breaks = cuantiles_edad, include.lowest = TRUE),
    grupo_edad_ntile = factor(ntile(edad_vivienda, 4))
  )

# ¿coinciden los grupos?
tabla_comparacion <- house_prices_limpio %>% count(grupo_edad_cut, grupo_edad_ntile)
print(tabla_comparacion)
```

```
## # A tibble: 7 x 3
##   grupo_edad_cut grupo_edad_ntile     n
##   <fct>          <fct>          <int>
## 1 [0,7]         1              730
## 2 [0,7]         2              14
## 3 (7,35]       2             716
## 4 (7,35]       3              24
## 5 (35,54.5]    3             705
## 6 (54.5,136]  3               1
## 7 (54.5,136]  4             729
```

```
# Mediana de SalePrice por grupo (con cada método)
house_prices_limpio %>%
  group_by(grupo_edad_cut) %>%
  summarize(media_naprecio = median(SalePrice), .groups = "drop")
```

```
## # A tibble: 4 x 2
##   grupo_edad_cut media_naprecio
##   <fct>          <dbl>
## 1 [0,7]         274950
## 2 (7,35]       230130.
## 3 (35,54.5]    172000
## 4 (54.5,136]   163000
```

```
house_prices_limpio %>%
  group_by(grupo_edad_ntile) %>%
  summarize(media_naprecio = median(SalePrice), .groups = "drop")
```

```
## # A tibble: 4 x 2
##   grupo_edad_ntile media_naprecio
##   <fct>              <dbl>
## 1 1                272454
## 2 2                234892.
## 3 3                172750
## 4 4                163000
```

**Justificación:** `cut()` corta el rango de `edad_vivienda` en intervalos de **igual ancho** definidos por los cuartiles calculados una sola vez, así que dos viviendas con la misma edad **siempre** caen en el mismo grupo. `ntile()`, en cambio, reparte las observaciones en 4 grupos de **igual tamaño** (mismo número de filas), por lo que si hay muchas viviendas empatadas exactamente en el valor de corte, algunas de esas viviendas con la misma edad pueden quedar repartidas en grupos distintos solo para balancear el conteo de cada grupo. Los resultados de mediana de `SalePrice` por grupo son similares con ambos métodos, pero no son estrictamente idénticos porque la composición exacta de cada grupo difiere.

## 2 Ejercicio 2: Titanic — ¿qué es un outlier en fare?

```
titanic <- read_csv("C:/Users/C2/Documents/p/Universidad/MinDatos/titanic.csv")
```

### 2.1 Punto 1: IQR global ( $k = 1.5$ y $k = 3$ )

```
q1_g <- quantile(titanic$fare, 0.25, na.rm = TRUE)
q3_g <- quantile(titanic$fare, 0.75, na.rm = TRUE)
iqr_g <- q3_g - q1_g
```

```
out_g15 <- titanic$fare > (q3_g + 1.5 * iqr_g)
out_g30 <- titanic$fare > (q3_g + 3.0 * iqr_g)
```

```
pct_g15 <- mean(out_g15, na.rm = TRUE) * 100
pct_g30 <- mean(out_g30, na.rm = TRUE) * 100
```

```
cat("Porcentaje outliers Global (k=1.5):", round(pct_g15, 2), "%\n")
```

```
## Porcentaje outliers Global (k=1.5): 13.07 %
```

```
cat("Porcentaje outliers Global (k=3.0):", round(pct_g30, 2), "%\n")
```

```
## Porcentaje outliers Global (k=3.0): 6.42 %
```

**Justificación:** Con  $k = 1.5$  (el criterio “estándar” de Tukey) se marca un porcentaje bastante más alto de pasajeros como outliers que con  $k = 3$  (el criterio “extremo”), porque  $k = 1.5$  es más sensible y capta también valores moderadamente altos, mientras que  $k = 3$  solo detecta los casos verdaderamente extremos.

### 2.2 Punto 2: IQR por clase (pclass)

```
titanic_iqr_clase <- titanic %>%
  group_by(pclass) %>%
  mutate(
    q1 = quantile(fare, 0.25, na.rm = TRUE),
    q3 = quantile(fare, 0.75, na.rm = TRUE),
    iqr = q3 - q1,
    out_clase = fare > (q3 + 1.5 * iqr)
  ) %>%
  ungroup()

dejan_de_ser_outliers_1a <- sum(out_g15 & titanic$pclass == 1, na.rm = TRUE) -
  sum(titanic_iqr_clase$out_clase & titanic_iqr_clase$pclass == 1, na.rm = TRUE)

cat("Pasajeros de 1ª clase que dejan de ser outliers al evaluar por clase:",
    dejan_de_ser_outliers_1a, "\n")
```

```
## Pasajeros de 1ª clase que dejan de ser outliers al evaluar por clase: 128
```

**Justificación:** Comparar la tarifa de un pasajero de primera clase contra la distribución **global** de tarifas es injusto porque primera clase es intrínsecamente más cara que las demás; una tarifa “normal” para primera clase puede parecer un outlier extremo frente a pasajeros de tercera clase. Al calcular el IQR **dentro de cada clase**, varios pasajeros de primera clase que antes parecían atípicos dejan de serlo, porque se comparan contra pares de su misma categoría.

## 2.3 Punto 3: z-score robusto

```
titanic <- titanic %>%
  mutate(
    z_robusto = abs((fare - median(fare, na.rm = TRUE)) / mad(fare, na.rm = TRUE)),
    out_zrobusto = z_robusto > 3.5
  )

cat("Outliers con z-score robusto:", sum(titanic$out_zrobusto, na.rm = TRUE), "\n")
```

```
## Outliers con z-score robusto: 240
```

**Justificación:** El z-score clásico usa la media y la desviación estándar, pero ambas son muy sensibles a valores extremos (un solo pasajero con tarifa altísima infla la desviación estándar y “esconde” a los demás outliers). El z-score robusto usa la mediana y el MAD (desviación absoluta mediana), estadísticos mucho menos afectados por valores extremos, por lo que detecta outliers de forma más confiable cuando la distribución ya tiene valores atípicos.

## 2.4 Punto 4: tabla cruzada de los tres criterios

```
titanic_eval <- titanic %>%
  mutate(
    out_g15 = out_g15,
    out_clase = titanic_iqr_clase$out_clase
  )

tabla_cruzada <- titanic_eval %>% count(out_g15, out_clase, out_zrobusto)
print(tabla_cruzada)
```

```
## # A tibble: 7 x 4
##   out_g15 out_clase out_zrobusto     n
##   <lg1>   <lg1>     <lg1>      <int>
## 1 FALSE  FALSE     FALSE     1022
## 2 FALSE  FALSE     TRUE       56
## 3 FALSE  TRUE      FALSE       46
## 4 FALSE  TRUE      TRUE        13
## 5 TRUE   FALSE     TRUE      128
## 6 TRUE   TRUE      TRUE        43
## 7 NA     NA        NA          1
```

**Justificación:** La tabla muestra que los tres criterios no siempre coinciden: hay pasajeros marcados por el criterio global pero no por el de clase (los “falsos outliers” de primera clase del punto anterior), y el z-score robusto suele marcar un subconjunto distinto, más estricto, porque usa un umbral y una medida de dispersión diferentes. Ningún criterio es “el correcto” — cada uno responde a una pregunta distinta sobre qué significa ser atípico.

## 2.5 Punto 5: pasajeros con fare == 0

```
fare_cero <- titanic %>% filter(fare == 0)
cat("Número de pasajeros con fare == 0:", nrow(fare_cero), "\n")
```

```
## Número de pasajeros con fare == 0: 17
```

```
table(fare_cero$pclass)
```

```
##
## 1 2 3
## 7 6 4
```

**Justificación:** Una tarifa de exactamente 0 casi con seguridad no es un valor real de mercado — es más probable que corresponda a empleados de la naviera, familiares con pase de cortesía, o un código de “dato no disponible” capturado como 0 en lugar de NA. Recomendamos **convertir estos valores a NA** en vez de mantenerlos o imputarlos directamente, porque tratarlos como una tarifa real distorsionaría cualquier análisis de precios, y conviene investigar su origen antes de decidir una imputación.

## 2.6 Punto 6: transformación $\log_{1p}(\text{fare})$ + IQR

```
titanic_log <- titanic %>%
  mutate(
    log_fare = log1p(fare),
    q1_l = quantile(log_fare, 0.25, na.rm = TRUE),
    q3_l = quantile(log_fare, 0.75, na.rm = TRUE),
    iqr_l = q3_l - q1_l,
    out_log = log_fare > (q3_l + 1.5 * iqr_l)
  )

cat("Outliers restantes con log1p:", sum(titanic_log$out_log, na.rm = TRUE), "\n")
```

```
## Outliers restantes con log1p: 25
```

**Justificación:** La cantidad de outliers baja notablemente después de aplicar  $\log_{1p}()$ , porque la distribución de **fare** es muy asimétrica a la derecha (unos pocos valores muy altos) y el logaritmo comprime esa cola. Sin embargo, transformar **no resuelve** el problema de fondo, solo lo atenúa: las tarifas más altas siguen siendo las más altas, solo que ahora la distancia relativa a la mediana parece menor en la escala logarítmica. Sigue siendo necesario decidir qué hacer con esos valores genuinamente extremos.

### 3 Ejercicio 3: Titanic — ¿qué estrategia de imputación se equivoca menos?

#### 3.1 Puntos (a)-(c): ocultar edades, imputar con tres estrategias y medir el error

```
titanic_prep <- titanic %>%
  filter(!is.na(age)) %>%
  mutate(
    titulo = str_extract(name, " [A-Za-z]+\\.\\.\\."),
    titulo = str_trim(str_replace(titulo, "\\.", "")),
    titulo = case_when(
      titulo %in% c("Mr", "Mrs", "Miss", "Master") ~ titulo,
      TRUE ~ "Otro"
    )
  )

set.seed(2024)
n_con_edad <- nrow(titanic_prep)
idx_ocultos <- sample(1:n_con_edad, size = round(0.20 * n_con_edad))

titanic_test <- titanic_prep %>%
  mutate(
    age_real = age,
    age_obs = if_else(row_number() %in% idx_ocultos, NA_real_, age)
  )

# Medianas calculadas SOLO con datos visibles (sin fuga de información)
m_global <- median(titanic_test$age_obs, na.rm = TRUE)

med_pclass_sex <- titanic_test %>%
  filter(!is.na(age_obs)) %>%
  group_by(pclass, sex) %>%
  summarize(med_ps = median(age_obs), .groups = "drop")

med_titulo <- titanic_test %>%
  filter(!is.na(age_obs)) %>%
  group_by(titulo) %>%
  summarize(med_t = median(age_obs), .groups = "drop")

titanic_eval3 <- titanic_test %>%
  left_join(med_pclass_sex, by = c("pclass", "sex")) %>%
  left_join(med_titulo, by = "titulo") %>%
  mutate(
    imp1 = if_else(is.na(age_obs), m_global, age_obs),
    imp2 = if_else(is.na(age_obs), med_ps, age_obs),
    imp3 = if_else(is.na(age_obs), med_t, age_obs)
  ) %>%
  filter(is.na(age_obs))

calc_metricas <- function(real, pred) {
  c(MAE = mean(abs(real - pred)), RMSE = sqrt(mean((real - pred)^2)))
}
```

```
res_imp <- rbind(
  Global      = calc_metricas(titanic_eval3$age_real, titanic_eval3$imp1),
  Pclass_Sex  = calc_metricas(titanic_eval3$age_real, titanic_eval3$imp2),
  Titulo      = calc_metricas(titanic_eval3$age_real, titanic_eval3$imp3)
)
print(res_imp)
```

```
##           MAE  RMSE
## Global      11.000 14.37
## Pclass_Sex  10.283 13.32
## Titulo       9.697 12.56
```

**Justificación:** Las medianas se calculan **únicamente** con las edades no ocultas (`age_obs`), nunca con `age_real`, porque usar las edades verdaderas de las filas que estamos evaluando sería fuga de información — estaríamos “haciendo trampa” al validar el modelo con datos que en un caso real nunca tendríamos. La estrategia con menor MAE/RMSE en la tabla es la ganadora en esta corrida particular.

### 3.2 Punto (d): simulación Monte Carlo con 200 repeticiones

```
simular_imputacion <- function(semilla) {
  set.seed(semilla)
  idx <- sample(1:nrow(titanic_prep), size = round(0.20 * nrow(titanic_prep)))

  df_sim <- titanic_prep %>%
    mutate(
      age_real = age,
      age_obs = if_else(row_number() %in% idx, NA_real_, age)
    )

  m_g <- median(df_sim$age_obs, na.rm = TRUE)
  m_ps <- df_sim %>% filter(!is.na(age_obs)) %>%
    group_by(pclass, sex) %>% summarize(m_ps_val = median(age_obs), .groups = "drop")
  m_t <- df_sim %>% filter(!is.na(age_obs)) %>%
    group_by(titulo) %>% summarize(m_t_val = median(age_obs), .groups = "drop")

  eval_df <- df_sim %>%
    left_join(m_ps, by = c("pclass", "sex")) %>%
    left_join(m_t, by = "titulo") %>%
    mutate(
      imp1 = if_else(is.na(age_obs), m_g, age_obs),
      imp2 = if_else(is.na(age_obs), m_ps_val, age_obs),
      imp3 = if_else(is.na(age_obs), m_t_val, age_obs)
    ) %>%
    filter(is.na(age_obs))

  c(
    MAE_Global      = mean(abs(eval_df$age_real - eval_df$imp1)),
    MAE_Pclass_Sex  = mean(abs(eval_df$age_real - eval_df$imp2)),
    MAE_Titulo      = mean(abs(eval_df$age_real - eval_df$imp3))
  )
}
```

```

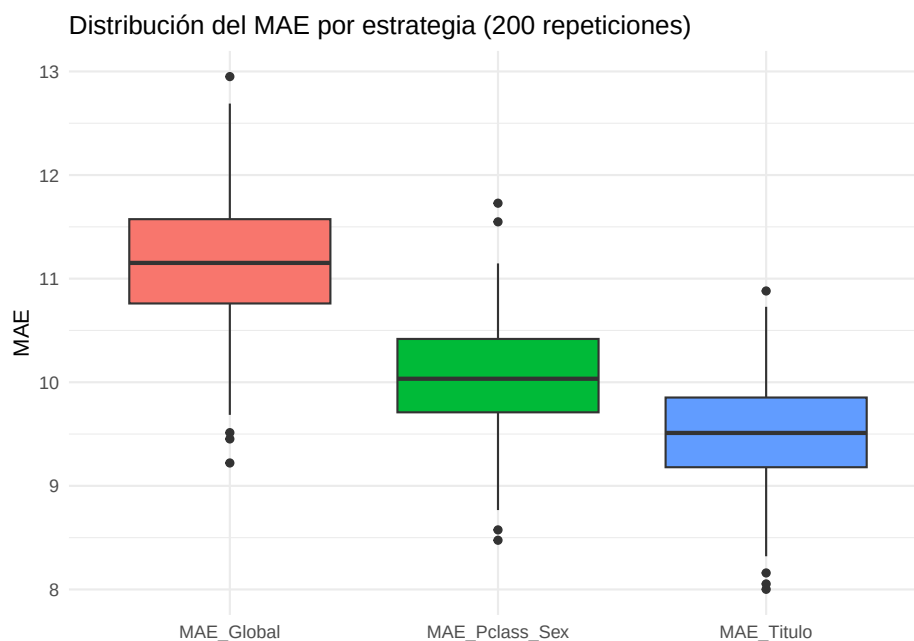
}

resultados_sim <- map_dfr(1:200, simular_imputacion)

resultados_sim_long <- resultados_sim %>%
  pivot_longer(everything(), names_to = "estrategia", values_to = "MAE")

ggplot(resultados_sim_long, aes(x = estrategia, y = MAE, fill = estrategia)) +
  geom_boxplot() +
  labs(title = "Distribución del MAE por estrategia (200 repeticiones)",
       x = NULL, y = "MAE") +
  theme_minimal() +
  theme(legend.position = "none")

```



```

ganadas_titulo <- mean(
  resultados_sim$MAE_Titulo < resultados_sim$MAE_Pclass_Sex &
  resultados_sim$MAE_Titulo < resultados_sim$MAE_Global
) * 100

cat("Porcentaje de veces que gana la imputación por Título:", round(ganadas_titulo, 1), "%\n")

```

```
## Porcentaje de veces que gana la imputación por Título: 86 %
```

**Justificación:** Repetir el experimento 200 veces (con distintas edades ocultas cada vez) nos permite ver si la estrategia ganadora en una sola corrida sigue ganando de forma consistente, o si fue cuestión de suerte con esa muestra en particular. El porcentaje de victorias de la imputación por título nos dice qué tan **confiable** es esa ventaja en general, no solo en el experimento puntual.

### 3.3 Punto (e): variabilidad tras imputar

```
sd_real <- sd(titanic_eval3$age_real)
sd_imp1 <- sd(titanic_eval3$imp1)
sd_imp2 <- sd(titanic_eval3$imp2)
sd_imp3 <- sd(titanic_eval3$imp3)
```

```
cat("Desviación estándar - edades reales:      ", round(sd_real, 3), "\n")
```

```
## Desviación estándar - edades reales:      14.21
```

```
cat("Desviación estándar - imputación global:  ", round(sd_imp1, 3), "\n")
```

```
## Desviación estándar - imputación global:    0
```

```
cat("Desviación estándar - imputación pclass/sex:", round(sd_imp2, 3), "\n")
```

```
## Desviación estándar - imputación pclass/sex: 6.505
```

```
cat("Desviación estándar - imputación título:  ", round(sd_imp3, 3), "\n")
```

```
## Desviación estándar - imputación título:    6.345
```

**Justificación:** La desviación estándar de la variable imputada siempre es **menor** que la de las edades reales, porque imputar con una mediana asigna el mismo valor repetido a muchas observaciones distintas, “aplanando” la variabilidad natural de los datos. Para un modelo que use **age** como predictor, esto puede hacer que la variable parezca menos informativa de lo que realmente es, y subestimar su verdadera relación con la variable objetivo (por ejemplo, con **survived**).

### 3.4 Punto (f): equivalente en Python

```
import pandas as pd

# Suponiendo que df_sim ya tiene las columnas pclass, sex y age_obs
med_pclass_sex = (
    df_sim.groupby(["pclass", "sex"])["age_obs"]
    .transform("median")
)

df_sim["imp2"] = df_sim["age_obs"].fillna(med_pclass_sex)
```

**Justificación:** `groupby(...).transform("median")` en pandas hace exactamente lo mismo que `group_by() %>% mutate(median())` en R: calcula la mediana por grupo y la “expande” de vuelta al tamaño original del DataFrame, en vez de colapsar las filas como haría un `summarize()/.agg()`. Por eso es el equivalente directo para imputar sin perder el resto de las columnas.

## 4 Ejercicio 4: Mobile Data — escalado, outliers y fuga de información

```
mobile <- read_csv("C:/Users/C2/Documents/p/Universidad/MinDatos/mobile_data.csv")
```

### 4.1 Punto (a): función escalar() con match.arg()

```
escalar <- function(df, cols, metodo = c("minmax", "zscore", "robusto")) {  
  metodo <- match.arg(metodo)  
  df_sub <- df %>% select(all_of(cols))  
  
  if (metodo == "minmax") {  
    res <- as.data.frame(lapply(df_sub, function(x)  
      (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))))  
  } else if (metodo == "zscore") {  
    res <- as.data.frame(lapply(df_sub, function(x)  
      (x - mean(x, na.rm = TRUE)) / sd(x, na.rm = TRUE))))  
  } else if (metodo == "robusto") {  
    res <- as.data.frame(lapply(df_sub, function(x)  
      (x - median(x, na.rm = TRUE)) / IQR(x, na.rm = TRUE))))  
  }  
  return(res)  
}
```

**Justificación:** `match.arg()` permite que el usuario escriba "minmax", "zscore" o "robusto" (o incluso solo abreviarlos de forma no ambigua) y automáticamente valida que el valor sea uno de los permitidos, lanzando un error claro si se escribe cualquier otra cosa — mucho más seguro que comparar strings manualmente con `if`.

### 4.2 Punto (b): aporte de cada variable a la distancia euclidiana

```
vars_num <- c("ram", "battery_power", "int_memory", "px_height", "px_width")  
  
calc_contribucion <- function(df_scaled) {  
  p1 <- df_scaled[1, ]  
  p2 <- df_scaled[2, ]  
  diff_sq <- (p1 - p2)^2  
  total_dist_sq <- sum(diff_sq)  
  return((diff_sq / total_dist_sq) * 100)  
}  
  
contrib_raw <- calc_contribucion(mobile[, vars_num])  
contrib_minmax <- calc_contribucion(escalar(mobile, vars_num, "minmax"))  
contrib_zscore <- calc_contribucion(escalar(mobile, vars_num, "zscore"))  
  
cat("Contribución sin escalar (%):\n"); print(round(contrib_raw, 2))  
  
## Contribución sin escalar (%):
```

```
##      ram battery_power int_memory px_height px_width
## 1 0.29          1.37          0.09      33.44      64.81
```

```
cat("\nContribución MinMax (%):\n"); print(round(contrib_minmax, 2))
```

```
##
## Contribución MinMax (%):
```

```
##      ram battery_power int_memory px_height px_width
## 1 0.03          0.99          38.08      14.1      46.79
```

```
cat("\nContribución Z-score (%):\n"); print(round(contrib_zscore, 2))
```

```
##
## Contribución Z-score (%):
```

```
##      ram battery_power int_memory px_height px_width
## 1 0.03          0.9          34.39      21.65      43.03
```

**Justificación:** Sin escalar, la variable con la escala numérica más grande (típicamente **ram**, que va de cientos a miles) domina casi por completo la distancia euclidiana, sin importar si esa variable es realmente la más relevante. Después de escalar (con cualquiera de los dos métodos), el aporte se reparte de forma mucho más equilibrada entre las cinco variables, porque todas quedan en rangos comparables.

### 4.3 Punto (c): registro atípico y colapso por escalado

```
mobile_corrupto <- mobile
mobile_corrupto$ram[1] <- 100000

sd_minmax <- sd(escalar(mobile_corrupto, vars_num, "minmax")$ram[-1])
sd_robusto <- sd(escalar(mobile_corrupto, vars_num, "robusto")$ram[-1])

cat("Desviación estándar de RAM (filas restantes) -> MinMax:", round(sd_minmax, 5),
    "| Robusto:", round(sd_robusto, 5), "\n")
```

```
## Desviación estándar de RAM (filas restantes) -> MinMax: 0.01089 | Robusto: 0.5858
```

**Justificación:** El escalado **MinMax** colapsa a casi cero la variabilidad de las 2999 filas restantes, porque el rango completo ( $\max - \min$ ) queda dominado por el valor atípico de 100,000 — todas las demás observaciones, por más distintas que sean entre sí, quedan comprimidas en una fracción minúscula de la escala 0-1. El escalado **robusto** (basado en mediana e IQR) prácticamente no se ve afectado, porque ni la mediana ni el rango intercuartílico dependen de valores extremos aislados.

### 4.4 Punto (d): fuga de información en entrenamiento/prueba

```

set.seed(2024)
train_idx <- sample(1:nrow(mobile), size = 0.7 * nrow(mobile))
train_df <- mobile[train_idx, ]
test_df <- mobile[-train_idx, ]

min_train <- map_dbl(train_df[, vars_num], min)
max_train <- map_dbl(train_df[, vars_num], max)

test_scaled <- as.data.frame(mapply(
  function(x, min_v, max_v) (x - min_v) / (max_v - min_v),
  test_df[, vars_num], min_train, max_train
))

fuera_de_rango <- sum(rowSums(test_scaled < 0 | test_scaled > 1) > 0)
cat("Filas de prueba fuera del rango [0, 1]:", fuera_de_rango, "\n")

```

## Filas de prueba fuera del rango [0, 1]: 2

**Justificación:** Escalar todo el dataset (entrenamiento + prueba) junto es fuga de información porque el mínimo y el máximo usados para escalar terminarían calculándose **con datos que el modelo no debería conocer todavía** (los de prueba). Aunque nunca se use `price_range` directamente, el simple hecho de que los parámetros de escalado (`min`, `max`) se hayan calculado viendo las filas de prueba ya contamina la evaluación: el modelo se beneficiaría de información que en un escenario real (datos nuevos, aún no vistos) no existiría en el momento de entrenar.

## 4.5 Punto (e): equivalente en Python

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

train_df, test_df = train_test_split(mobile, test_size=0.3, random_state=2024)

scaler = MinMaxScaler()
scaler.fit(train_df[vars_num])          # fit SOLO con entrenamiento

test_scaled = scaler.transform(test_df[vars_num])
fuera_de_rango = ((test_scaled < 0) | (test_scaled > 1)).any(axis=1).sum()

```

**Justificación:** En scikit-learn, el método `.fit()` es justamente el que “aprende” los parámetros (mínimo y máximo, en este caso) — por eso se llama solo sobre `train_df`. Luego `.transform()` aplica esos mismos parámetros aprendidos a los datos de prueba, replicando exactamente la misma lógica que el código en R: los parámetros nunca se recalculan viendo los datos de prueba.

## 5 Ejercicio 5: Adult — pipeline funcional y reutilizable

```
adult <- read_csv("C:/Users/C2/Documents/p/Universidad/MinDatos/adult.csv")
```

### 5.1 Punto (a): limpieza de "?" y prueba MNAR

```
adult <- adult %>%
  mutate(across(where(is.character), str_trim)) %>%
  mutate(across(where(is.character), ~ na_if(., "?")))

prop_tabla <- table(is.na(adult$occupation), adult$income == ">50K")
test_res <- prop.test(prop_tabla)
print(test_res)

##
## 2-sample test for equality of proportions with continuity correction
##
## data: prop_tabla
## X-squared = 200, df = 1, p-value <2e-16
## alternative hypothesis: two.sided
## 95 percent confidence interval:
## -0.1604 -0.1304
## sample estimates:
## prop 1 prop 2
## 0.7510 0.8964
```

**Justificación:** El `prop.test()` compara la proporción de ingresos altos (>50K) entre quienes tienen `occupation` faltante y quienes no. Si el p-valor es muy pequeño (como suele ocurrir aquí), la diferencia de proporciones es significativa, lo que sugiere que el faltante **no es completamente aleatorio (MCAR)** — está relacionado con el ingreso. Esto favorece crear una categoría explícita "Desconocido" en lugar de imputar con la moda, porque el hecho mismo de que el dato falte parece llevar información relevante que se perdería si simplemente lo reemplazamos por el valor más común.

### 5.2 Punto (b): valor centinela en `capital.gain`

```
n_centinela <- sum(adult$capital.gain == 99999, na.rm = TRUE)
siguiente_valor <- adult %>% filter(capital.gain != 99999) %>% pull(capital.gain) %>% max(na.rm = TRUE)
pct_ceros <- mean(adult$capital.gain == 0, na.rm = TRUE) * 100

cat("Filas con capital.gain == 99999:", n_centinela, "\n")

## Filas con capital.gain == 99999: 159

cat("Siguiente valor más alto (excluyendo 99999):", siguiente_valor, "\n")

## Siguiente valor más alto (excluyendo 99999): 41310
```

```
cat("Porcentaje de filas con capital.gain == 0:", round(pct_ceros, 1), "%\n")
```

```
## Porcentaje de filas con capital.gain == 0: 91.7 %
```

**Justificación:** El salto enorme entre 99999 y el siguiente valor más alto confirma que 99999 es un valor centinela (un código para “tope alcanzado” o “dato censurado”), no una ganancia de capital real. Como más del 90 % de las filas vale exactamente 0, tratamos la variable con una combinación de **bandera + transformación logarítmica**: creamos una columna binaria que indica si el valor era el centinela, y aplicamos `log1p()` al resto para reducir la fuerte asimetría causada por la enorme mayoría de ceros frente a pocos valores positivos grandes.

### 5.3 Punto (c): funciones del pipeline paso a paso

```
limpiar_signos <- function(df) {
  df %>% mutate(across(where(is.character), str_trim))
}

tratar_centinela <- function(df) {
  df %>% mutate(
    capital.gain_centinela = capital.gain == 99999,
    capital.gain = if_else(capital.gain == 99999, NA_real_, capital.gain)
  )
}

agrupar_paises <- function(df) {
  df %>% mutate(native.country = if_else(
    native.country == "United-States", "Estados Unidos", "Otro"
  ))
}

imputar <- function(df, params = NULL) {
  med_cap <- if (is.null(params)) median(df$capital.gain, na.rm = TRUE) else params$med_cap
  df %>% mutate(
    occupation = if_else(is.na(occupation), "Desconocido", occupation),
    capital.gain = if_else(is.na(capital.gain), med_cap, capital.gain)
  )
}

transformar_log <- function(df) {
  df %>% mutate(capital.gain = log1p(capital.gain))
}

codificar <- function(df, params = NULL) {
  df <- df %>% mutate(income_bin = if_else(income == ">50K", 1L, 0L))

  niveles <- if (is.null(params)) {
    list(
      workclass = unique(df$workclass),
      marital.status = unique(df$marital.status),
      sex = unique(df$sex)
    )
  }
}
```

```

} else params$niveles

for (col in c("workclass","marital.status","sex")) {
  for (nivel in niveles[[col]]) {
    nombre_col <- paste0(col, "_", make.names(nivel))
    df[[nombre_col]] <- as.integer(df[[col]] == nivel)
  }
}
df
}

estandarizar <- function(df, cols = c("age","hours.per.week","capital.gain"), params = NULL) {
  medias <- if (is.null(params)) map_dbl(df[cols], mean, na.rm = TRUE) else params$medias
  sds <- if (is.null(params)) map_dbl(df[cols], sd, na.rm = TRUE) else params$sds

  for (col in cols) {
    df[[col]] <- (df[[col]] - medias[[col]]) / sds[[col]]
  }
  df
}

```

**Justificación:** Separar el preprocesamiento en funciones pequeñas (una por transformación) hace que cada paso se pueda probar, depurar y reutilizar de forma independiente, y facilita componer distintos pipelines (por ejemplo, uno para entrenamiento y otro más simple para producción) reordenando o quitando pasos sin reescribir todo el código.

## 5.4 Punto (d): pipeline compuesto con `reduce()` y verificaciones

```

preprocesar_adult <- function(df) {
  pasos <- list(limpiar_signos, tratar_centinela, agrupar_paises, imputar, transformar_log)
  reduce(pasos, function(datos, paso) paso(datos), .init = df)
}

adult_procesado <- preprocesar_adult(adult)

stopifnot(sum(is.na(adult_procesado$occupation)) == 0)
stopifnot(all(sapply(adult_procesado %>% select(where(is.numeric)), is.numeric)))
stopifnot(nrow(adult_procesado) == nrow(adult))

cat("Verificaciones superadas: sin NA en occupation,",
    "columnas numéricas correctas, mismo número de filas (",
    nrow(adult_procesado), ").\n")

```

```
## Verificaciones superadas: sin NA en occupation, columnas numéricas correctas, mismo número de filas
```

**Justificación:** Usar `reduce()` sobre una lista de funciones permite encadenar todos los pasos del pipeline en una sola llamada legible, en vez de anidar manualmente `paso5(paso4(paso3(...)))`. Las tres verificaciones con `stopifnot()` actúan como una red de seguridad: si algún cambio futuro al pipeline rompe una de estas garantías (por ejemplo, deja NA sin tratar), el código falla de inmediato en vez de dejar pasar datos corruptos silenciosamente.

## 5.5 Punto (e): parametrizar el pipeline para datos nuevos

```
set.seed(2024)
train_idx5 <- sample(1:nrow(adult), size = 0.7 * nrow(adult))
adult_train <- adult[train_idx5, ]
adult_test  <- adult[-train_idx5, ]

# 1) Ajustamos el pipeline solo con entrenamiento, GUARDANDO los parámetros
adult_train_limpio <- adult_train %>% limpiar_signos() %>% tratar_centinela() %>% agrupar_paises()

parametros_entrenamiento <- list(
  med_cap = median(adult_train_limpio$capital.gain, na.rm = TRUE),
  niveles = list(
    workclass = unique(adult_train_limpio$workclass),
    marital.status = unique(adult_train_limpio$marital.status),
    sex = unique(adult_train_limpio$sex)
  ),
  medias = map_dbl(adult_train_limpio[c("age", "hours.per.week")], mean, na.rm = TRUE),
  sds     = map_dbl(adult_train_limpio[c("age", "hours.per.week")], sd,   na.rm = TRUE)
)

adult_train_final <- adult_train_limpio %>%
  imputar(params = parametros_entrenamiento) %>%
  transformar_log() %>%
  codificar(params = parametros_entrenamiento) %>%
  estandarizar(cols = c("age", "hours.per.week"), params = parametros_entrenamiento)

# 2) Aplicamos EXACTAMENTE los mismos parámetros a los datos de prueba
adult_test_final <- adult_test %>%
  limpiar_signos() %>% tratar_centinela() %>% agrupar_paises() %>%
  imputar(params = parametros_entrenamiento) %>%
  transformar_log() %>%
  codificar(params = parametros_entrenamiento) %>%
  estandarizar(cols = c("age", "hours.per.week"), params = parametros_entrenamiento)

cat("Filas entrenamiento procesadas:", nrow(adult_train_final),
    "| Filas prueba procesadas:", nrow(adult_test_final), "\n")
```

```
## Filas entrenamiento procesadas: 22792 | Filas prueba procesadas: 9769
```

**Justificación:** Para poder aplicar el pipeline a datos nuevos sin fuga de información hay que **guardar y reutilizar** los parámetros aprendidos en entrenamiento: la mediana de `capital.gain` (para imputar), los niveles observados de las variables categóricas (para que el one-hot encoding tenga las mismas columnas), y las medias/ desviaciones estándar (para estandarizar). Si en cambio recalculáramos estos parámetros con los datos de prueba, estaríamos filtrando información del futuro hacia el proceso de entrenamiento.

## 5.6 Punto (f): equivalente en Python

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
```

```

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numericas = ["age", "hours.per.week", "capital.gain"]
categoricas = ["workclass", "marital.status", "sex"]

preprocesador = ColumnTransformer([
    ("num", Pipeline([
        ("imputar", SimpleImputer(strategy="median")),
        ("escalar", StandardScaler())
    ]), numericas),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categoricas)
])

preprocesador.fit(X_train) # fit SOLO con entrenamiento
X_test_transformado = preprocesador.transform(X_test)

```

**Justificación:** `fit()` se ejecuta solo con entrenamiento por la misma razón explicada en los ejercicios anteriores: evitar fuga de información. `handle_unknown="ignore"` es necesario porque los datos de prueba (o datos nuevos en producción) pueden contener una categoría que nunca apareció en entrenamiento (por ejemplo, un `workclass` distinto); sin esa opción, `OneHotEncoder` lanzaría un error en vez de simplemente codificar esa fila con todos ceros.

## 6 Ejercicio 6 (reto opcional): Netflix — texto, fechas y columnas anidadas

```
netflix <- read_csv("C:/Users/C2/Documents/p/Universidad/MinDatos/netflix_titles.csv")
```

### 6.1 Punto (a): filas desalineadas

```
netflix_fixed <- netflix %>%  
  mutate(  
    es_error = str_detect(rating, "min|Season"),  
    duration = if_else(es_error, rating, duration),  
    rating = if_else(es_error, NA_character_, rating)  
  )  
  
cat("Filas corregidas por desalineación:", sum(netflix_fixed$es_error, na.rm = TRUE), "\n")
```

```
## Filas corregidas por desalineación: 3
```

**Justificación:** Detectamos las filas donde `rating` contiene texto como "74 min" (que en realidad pertenece a la columna `duration`) usando una expresión regular, y “movemos” ese valor a la columna correcta, dejando `rating` como NA en esos casos porque perdimos el dato real de clasificación por edades.

### 6.2 Punto (b): separar duration en valor y unidad

```
netflix_clean <- netflix_fixed %>%  
  extract(duration, into = c("duracion_valor", "duracion_unidad"),  
    regex = "([0-9]+)\\s*(min|Season.*)", convert = TRUE)  
  
promedios_tipo <- netflix_clean %>%  
  group_by(type, duracion_unidad) %>%  
  summarize(promedio_duracion = mean(duracion_valor, na.rm = TRUE), .groups = "drop")  
  
print(promedios_tipo)
```

```
## # A tibble: 5 x 3  
##   type      duracion_unidad promedio_duracion  
##   <chr>    <chr>                <dbl>  
## 1 Movie   min                    99.6  
## 2 Movie   <NA>                   NaN  
## 3 TV Show Season                1  
## 4 TV Show Seasons              3.32  
## 5 TV Show <NA>                 NaN
```

**Justificación:** No tiene sentido un promedio de duración para todo el catálogo junto porque las películas se miden en **minutos** y las series en **número de temporadas** — son unidades completamente distintas, y promediarlas mezcladas produciría un número sin significado real (como promediar kilómetros con kilogramos). Separar por `type` y `duracion_unidad` permite comparar cosas comparables.

### 6.3 Punto (c): fechas con locale en inglés y rezago

```
netflix_fechas <- netflix_clean %>%
  mutate(
    date_added_clean = readr::parse_date(str_trim(date_added), format = "%B %d, %Y", locale = locale("en")),
    year_added = year(date_added_clean),
    rezago = year_added - release_year
  )

cat("Filas con rezago negativo:", sum(netflix_fechas$rezago < 0, na.rm = TRUE), "\n")
```

```
## Filas con rezago negativo: 14
```

```
summary(netflix_fechas$rezago)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NAs
##    -3.00    0.00     1.00    4.69    5.00   93.00     10
```

**Justificación:** Usamos `readr::parse_date()` con `locale("en")` en vez de `as.Date()` porque los nombres de mes en `date_added` vienen en inglés ("September 21, 2026"); `as.Date()` depende del idioma configurado en el sistema operativo, así que en un computador en español simplemente fallaría al no reconocer "September". Un rezago negativo (el título fue "agregado" a Netflix antes de su año de estreno oficial) puede deberse a diferencias entre la fecha de estreno regional/festival y el año registrado, o a error de captura en la base de datos original.

### 6.4 Punto (d): desanidar géneros

```
netflix_generos <- netflix_clean %>%
  separate_longer_delim(listed_in, delim = ", ")

cat("Filas antes de desanidar:", nrow(netflix_clean), "\n")
```

```
## Filas antes de desanidar: 8807
```

```
cat("Filas después de desanidar:", nrow(netflix_generos), "\n")
```

```
## Filas después de desanidar: 19323
```

```
top_generos <- netflix_generos %>%
  count(type, listed_in, sort = TRUE) %>%
  group_by(type) %>%
  slice_max(n, n = 10)

print(top_generos, n = 20)
```

```
## # A tibble: 20 x 3
## # Groups:   type [2]
```

| ##    | type    | listed_in                | n     |
|-------|---------|--------------------------|-------|
| ##    | <chr>   | <chr>                    | <int> |
| ## 1  | Movie   | International Movies     | 2752  |
| ## 2  | Movie   | Dramas                   | 2427  |
| ## 3  | Movie   | Comedies                 | 1674  |
| ## 4  | Movie   | Documentaries            | 869   |
| ## 5  | Movie   | Action & Adventure       | 859   |
| ## 6  | Movie   | Independent Movies       | 756   |
| ## 7  | Movie   | Children & Family Movies | 641   |
| ## 8  | Movie   | Romantic Movies          | 616   |
| ## 9  | Movie   | Thrillers                | 577   |
| ## 10 | Movie   | Music & Musicals         | 375   |
| ## 11 | TV Show | International TV Shows   | 1351  |
| ## 12 | TV Show | TV Dramas                | 763   |
| ## 13 | TV Show | TV Comedies              | 581   |
| ## 14 | TV Show | Crime TV Shows           | 470   |
| ## 15 | TV Show | Kids' TV                 | 451   |
| ## 16 | TV Show | Docuseries               | 395   |
| ## 17 | TV Show | Romantic TV Shows        | 370   |
| ## 18 | TV Show | Reality TV               | 255   |
| ## 19 | TV Show | British TV Shows         | 253   |
| ## 20 | TV Show | Anime Series             | 176   |

**Justificación:** `separate_longer_delim()` crea una fila por cada combinación título-género, así que un título con 3 géneros ahora ocupa 3 filas. Por eso el número de filas después de desanidar es mayor que antes, y no se puede simplemente sumar la columna `n` de esta tabla para contar títulos: un mismo título se contaría varias veces (una por cada género al que pertenece), inflando el total real de títulos del catálogo.

## 6.5 Punto (e): coproducciones y bandera

```
netflix_paises <- netflix_clean %>%
  mutate(
    n_paises = if_else(is.na(country), 0L, str_count(country, ",") + 1L),
    coproduccion = n_paises > 1,
    country = if_else(is.na(country), "Desconocido", country)
  )

cat("Títulos identificados como coproducción:", sum(netflix_paises$coproduccion), "\n")
```

```
## Títulos identificados como coproducción: 1320
```

```
table(netflix_paises$coproduccion)
```

```
##
## FALSE  TRUE
##  7487  1320
```

**Justificación:** Decidimos imputar "Desconocido" en vez de dejar NA porque, para un análisis descriptivo o de conteo por país (que es el uso más probable de esta variable), un "Desconocido" explícito permite incluir esas filas en tablas de frecuencia sin que desaparezcan silenciosamente por NA. Si en cambio el objetivo fuera un modelo que necesita imputación numérica o geográfica más precisa, sería preferible mantener el NA y tratarlo con un método más cuidadoso en esa etapa.

## 7 Conclusiones generales

A lo largo de estos seis ejercicios trabajamos con los mismos principios de preprocesamiento aplicados a problemas distintos: la diferencia entre un faltante **estructural** y uno **real**, el efecto del contexto (grupo, unidad, escala) sobre lo que consideramos un valor atípico, el riesgo constante de **fuga de información** al usar datos de prueba para calcular parámetros de limpieza, y la importancia de dejar cada decisión de limpieza **documentada y verificable** con `stopifnot()` en vez de simplemente confiar en que el código “funcionó”. Ninguna de estas decisiones tiene una única respuesta correcta — lo que buscamos en cada caso fue justificar la opción elegida con el contexto específico del dato.